

SAD · UNIDAD 6

Bastionado: cerrar puertas y ventanas

Quitar la llave maestra, poner cerrojos, un portero y un inspector

LA SUPERFICIE DE ATAQUE

Un servidor recién instalado… y descuidado

root entra por SSH con contraseña	`sshpass -p bodega2026 ssh root@10.10.20.30 whoami → root`
Nginx dice su versión exacta	`curl -I http://10.10.20.30 → Server: nginx/1.26.3`
Una cuenta «soporte» con UID 0 (= otro root)	`soporte:x:0:0::/root:/bin/bash`
Un programa con SUID que nadie conoce	`/usr/local/bin/copia-seguridad`
Fail2ban instalado… sin ninguna cárcel	`Number of jail: 0`

SUPERFICIE DE ATAQUE — Todas las puertas y ventanas por las que se podría entrar. Bastionar = reducirla.

CÓMO SE HACE

Medir, cambiar, volver a medir

1

Medir

El inspector revisa y pone nota

`./lab.sh lynis`

2

Cambiar

Una cosa cada vez, y probar que sigue funcionando

`./lab.sh ssh · nginx · fail2ban`

3

Volver a medir

¿Ha subido la nota? ¿He roto algo?

`./lab.sh lynis · comprobar ud6`

NUNCA TODO DE GOLPE — Si cambias diez cosas y algo se rompe, no sabrás cuál ha sido.

Paso 1: medir

```
./lab.sh lynis   (el servidor como arranca)

Nota del servidor (hardening index): 58 / 100
Avisos (warning): lo más grave
  TOOL-5104 · All jails in Fail2ban are disabled
Sugerencias de lo que se toca en la UD6:
  SSH-7408 · … · MaxAuthTries (set 6 to 3)
  SSH-7408 · … · PermitRootLogin (set YES to …NO…)
  SSH-7408 · … · X11Forwarding (set YES to NO)
  SSH-7408 · … · AllowTcpForwarding (set YES to NO)
  HTTP-6710 · Add HTTPS to nginx virtual hosts…
  KRNL-6000 · One or more sysctl values differ…
(en total: 49 sugerencias; todas en /var/log/lynis-report.dat)
```

58

hardening index: la nota, de 0 a 100

warning

aviso: lo grave: arréglalo

suggestion

sugerencia: valora si te aplica

LA NOTA ES UNA GUÍA — No hay que llegar a 100: hay que entender cada aviso y decidir.

LA PUERTA PRINCIPAL: SSH

Qué es y por qué es la que más importa

¿Qué es SSH?

Entrar a un servidor por la red y manejarlo con el teclado, desde cualquier sitio. Es LA puerta de entrada.

El problema de serie

Deja entrar a root y con contraseña, y a esa puerta llaman bots de todo Internet probando sin parar.

Lo que vamos a hacer

root no entra directo: entras como usuario normal y pides permiso (sudo). Así queda apuntado quién ha sido.

una llave en vez de una contraseña: la contraseña se adivina probando millones; la llave (UD4), no.

ANALOGÍA — La puerta del edificio: hoy cualquiera prueba llaves sin parar y, si acierta, es el dueño.

Cuatro cerrojos en un fichero propio

servidor/ssh/bastionado.conf

```
PermitRootLogin no
PasswordAuthentication no
MaxAuthTries 3
X11Forwarding no
```

`PermitRootLogin no`

root no entra: usuario normal + sudo

`PasswordAuthentication no`

solo con llave (UD4): no se adivina

`MaxAuthTries 3`

como mucho 3 intentos por conexión

`X11Forwarding no`

ventanas gráficas: en un servidor, no

¿Dónde?

En /etc/ssh/sshd_config.d/ un fichero que acabe en .conf. No se toca sshd_config.

Manda el primero

SSH se queda con el PRIMER valor que lee: sshd_config.d/ se lee antes.

DESPUÉS — ./lab.sh ssh → comprueba (sshd -t) y, si está bien, aplica.

DEMO · SSH

La misma orden, antes y después

ANTES · ./lab.sh internet

```
sshpass -p bodega2026 ssh root@10.10.20.30 whoami
root
```

DESPUÉS · ./lab.sh internet

```
sshpass -p bodega2026 ssh root@10.10.20.30 whoami
root@10.10.20.30: Permission denied (publickey).
```

ANTES · sshd -T (lo que vale de verdad)

```
permitrootlogin yes
passwordauthentication yes
maxauthtries 6
x11forwarding yes
```

DESPUÉS · sshd -T

```
permitrootlogin no
passwordauthentication no
maxauthtries 3
x11forwarding no
```

(publickey) = solo admite llave: ni pregunta la contraseña

Y el administrador, con su llave: `ssh admin@10.10.20.30 whoami` → `admin`

sshd -T — Enseña lo que SSH está usando de verdad, no lo que crees que pusiste.

NO TE QUEDES FUERA

El orden seguro para cerrar las contraseñas

1

Copia tu llave

```
ssh-copy-id
admin@servidor
```

2

Prueba con la llave

```
en OTRA terminal
```

3

Cierra contraseñas

```
PasswordAuthentication
no
```

4

Vuelve a probar

```
sin cerrar la sesión
abierta
```

Si te saltas el paso 2

Cierras las contraseñas sin llave en el servidor → nadie vuelve a entrar. Ni tú.

EN EL COMPROBADOR — «El administrador sigue entrando con su llave (no te has dejado fuera)».

Qué es y por qué se ajusta

El núcleo (kernel)

El programa central: reparte memoria, red y procesador entre todos. No se ve, pero manda en todo.

sysctl

Los «mandos» del núcleo: ajustan cómo se comporta, sin reiniciar.

¿Por qué tocarlo? Unos pocos mandos le ponen muy difícil a un atacante aprovechar un fallo de un programa: no lo arreglan, pero lo vuelven casi imposible de explotar.

ANALOGÍA — El motor del coche: ajustes de fábrica cómodos pero poco seguros. Cambiamos solo tres que importan.

EL NÚCLEO: SYSCTL

Tres ajustes que importan

servidor/sysctl.d/99-bastionado.conf

```
kernel.randomize_va_space = 2
kernel.dmesg_restrict = 1
net.ipv4.conf.all.accept_redirects = 0
```

ASLR completo

Cada programa, en un sitio de la memoria al azar: un fallo es más difícil de aprovechar.

dmesg solo para root

Los mensajes del núcleo dan pistas al atacante.

Sin redirecciones ICMP

Que nadie de la red nos desvíe el tráfico.

EN UN SERVIDOR DE VERDAD — sysctl --system lo aplica. En el laboratorio el núcleo es el de tu ordenador: solo se revisa el fichero.

UID 0 y SUID, explicados despacio

UID 0 = root, se llame como se llame

```
root    …:0:0:… /root
soporte …:0:0:… /root
        └┬┘
        el número manda
```

El sistema mira el NÚMERO (0), no el nombre. «soporte» tiene UID 0 → es root con otro nombre.

SUID = ejecutar como el dueño

Un recado firmado por el jefe: lo lleve quien lo lleve, vale como si lo diera el jefe.

- Se ejecuta con los permisos de su DUEÑO (casi siempre root), lo lance quien lo lance.

passwd lo necesita (escribe en /etc/shadow, que es de root).

uno desconocido en /usr/local = cualquiera se vuelve root: puerta trasera.

LA REGLA — UID 0 solo root. SUID: los del sistema (passwd, su, mount) valen; el que no conoces, fuera.

Llaves maestras escondidas

¿quién tiene UID 0?

```
awk -F: '$3==0' /etc/passwd
root:x:0:0:root:/root:/bin/bash
soporte:x:0:0::/root:/bin/bash
```

UID 0 = root

El sistema mira el número, no el nombre.

¿qué programas tienen SUID?

```
find / -xdev -perm -4000 -type f 2>/dev/null
/usr/lib/openssh/ssh-keysign
/usr/local/bin/copia-seguridad
/usr/bin/passwd
/usr/bin/su
/usr/bin/mount  …
```

Arreglo

Borrar la cuenta que sobra (userdel) o quitarle el UID 0.

SUID que sobra

Se ejecuta como root lo lance quien lo lance. En /usr/local y desconocido: fuera (chmod u-s).

SUID NORMAL — passwd, su o mount lo necesitan: son del sistema. El que no conoces es el sospechoso.

EL SERVICIO WEB HABLA DE MÁS

Qué son las cabeceras y por qué importan

Cuenta de más

Por defecto dice su versión exacta. El atacante busca los fallos conocidos de ESA versión.

Puede proteger

El servidor manda instrucciones de seguridad al navegador para que cuide al usuario.

¿Qué es una cabecera? Datos ocultos que el navegador lee ANTES de enseñar la página. No arreglan fallos, pero quitan pistas y piden ayuda al navegador.

ANALOGÍA — Como la etiqueta de un producto: le dice al navegador cómo tratar la página antes de usarla.

NGINX

Que no diga su versión y que mande cabeceras de seguridad

servidor/nginx/seguridad.conf

```
server_tokens off;
add_header X-Frame-Options "DENY" always;
add_header X-Content-Type-Options "nosniff" always;
add_header Content-Security-Policy "default-src 'self'" always;
```

Línea	Para qué
`server_tokens off`	No dice la versión: el atacante no sabe qué fallos buscar
`X-Frame-Options DENY`	Nadie mete nuestra web en un marco para engañar con clics (clickjacking)
`X-Content-Type-Options nosniff`	El navegador no adivina el tipo de un fichero
`Content-Security-Policy`	Solo se carga lo de nuestra web: frena el XSS (UD5)
`always`	También en las páginas de error (404, 500…)

DESPUÉS — ./lab.sh nginx → comprueba (nginx -t) y recarga.

DEMO · NGINX

curl -I: lo que el servidor cuenta de sí mismo

ANTES · curl -I http://10.10.20.30

```
HTTP/1.1 200 OK
Server: nginx/1.26.3
Date: Wed, 28 Oct 2026 22:14:16 GMT
Content-Type: text/html
Content-Length: 31
Connection: keep-alive
Accept-Ranges: bytes
```

DESPUÉS · curl -I http://10.10.20.30

```
HTTP/1.1 200 OK
Server: nginx
Date: Wed, 28 Oct 2026 22:15:27 GMT
Content-Type: text/html
Content-Length: 31
Connection: keep-alive
X-Frame-Options: DENY
X-Content-Type-Options: nosniff
Content-Security-Policy: default-src 'self'
Accept-Ranges: bytes
```

SE MIRA DESDE FUERA — ./lab.sh internet: lo que ve el atacante es lo que cuenta.

EL PROBLEMA: LA FUERZA BRUTA

Por qué un buen cerrojo no basta

Fuerza bruta

Un bot prueba miles de usuarios y contraseñas sin parar contra tu SSH, día y noche.

El cerrojo no basta

MaxAuthTries 3 corta CADA conexión a 3 intentos… pero el bot abre otra, y otra: puede probar millones.

La solución: un portero con memoria

Fail2ban apunta quién falla una y otra vez y, si insiste, le cierra la puerta (una regla en nftables, UD3). El cerrojo frena un empujón; el portero reconoce al pesado y no le deja ni acercarse.

ANALOGÍA — El cerrojo detiene un empujón. El portero reconoce al pesado y no le deja ni acercarse.

FAIL2BAN: EL PORTERO

Apunta al que prueba llaves y lo deja fuera

1 **Lee el registro**
`/var/log/auth.log`

→

2 **Busca los fallos**
`el filtro de SSH`

→

3 **Cuenta por IP**
`maxretry en findtime`

→

4 **Banea**
`regla en nftables (UD3)`

servidor/fail2ban/jail.local

```
[sshd]
enabled  = true
logpath  = /var/log/auth.log
maxretry = 3
findtime = 10m
bantime  = 1h
```

cárcel (jail) = la configuración para un servicio

maxretry · findtime = 3 fallos en 10 minutos…

bantime = …1 hora fuera

jail.local, nunca jail.conf (se machaca al actualizar)

DESPUÉS — ./lab.sh fail2ban → comprueba y recarga. Cárceles: sshd

DEMO · FAIL2BAN

Cinco intentos… y fuera

./lab.sh internet (5 usuarios inventados)

```
ssh pepe@10.10.20.30
pepe@10.10.20.30: Permission denied (publickey).
ssh maria@10.10.20.30
maria@10.10.20.30: Permission denied (publickey).
ssh test@10.10.20.30
ssh: connect to host 10.10.20.30 port 22: Connection refused
```

fail2ban-client status sshd

```
Status for the jail: sshd
|- Filter
|  `- Total failed:     7
`- Actions
   |- Currently banned: 1
   `- Banned IP list:   203.0.113.50
```

/var/log/fail2ban.log

```
[sshd] Found 203.0.113.50 - 22:15:30
[sshd] Found 203.0.113.50 - 22:15:30
[sshd] Ban 203.0.113.50
```

nft list ruleset (lo que ha puesto)

```
set addr-set-sshd {
    elements = { 203.0.113.50 }
}
tcp dport 22 ip saddr @addr-set-sshd reject
```

QUITAR EL CASTIGO — fail2ban-client unban --all · en la vida real, tu IP en ignoreip para no banearte.

VOLVER A MEDIR

Paso 3: ¿ha subido la nota?

58

antes

→

63

después

```
./lab.sh lynis   (después)

Nota del servidor (hardening index): 63 / 100
Avisos (warning): lo más grave
  ninguno
(en total: 46 sugerencias)
```

Ha desaparecido

El aviso de Fail2ban y las sugerencias de root, intentos y X11.

¿Por qué no 100?

Revisa cosas que aquí no tocamos (particiones, paquetes, HTTPS) o que no aplican en un contenedor.

LO IMPORTANTE — No el número: que cada aviso que queda esté entendido y decidido.

LAS GUÍAS

Bastionar es seguir una guía, no improvisar

CIS Benchmarks

Internacionales y gratuitas. Una por sistema (Debian, Nginx, SSH...).
Nivel 1: lo básico, sin romper nada. Nivel 2: más estricto.

CCN-STIC

Centro Criptológico Nacional (España).
Las siguen las administraciones públicas para cumplir el ENS.

En una empresa: elegir una guía → aplicarla → apuntar lo que NO se aplica y por qué → medir (Lynis).

ENS — Esquema Nacional de Seguridad: obligatorio en la administración pública española.

ERRORES TÍPICOS

Los cuatro que más vais a ver

Cerrar contraseñas sin llave

Nadie vuelve a entrar. Ni tú.

✗ `PasswordAuthentication no  (sin probar)`

✓ `copiar llave → probar → cerrar`

Un fichero que no acaba en .conf

SSH (o Nginx) no lo lee

✗ `sshd_config.d/bastionado.txt`

✓ `sshd_config.d/bastionado.conf`

La cárcel sin activar

Fail2ban arrancado… sin portero

✗ `[sshd]  enabled = false`

✓ `[sshd]  enabled = true  ·  maxretry = 3`

Cabeceras sin always

No se mandan en las páginas de error

✗ `add_header X-Frame-Options "DENY";`

✓ `add_header X-Frame-Options "DENY" always;`

CONTROL DE COMPRENSIÓN

Preguntas de control y debate

1. ¿Por qué PermitRootLogin no, si root es el administrador?
2. Una cuenta «soporte» tiene UID 0. ¿Qué problema hay?
3. ¿Qué significa que un programa tenga SUID? ¿Cuándo es sospechoso?
4. Explica, paso a paso, cómo Fail2ban deja fuera a una IP.
5. Lynis da 63. ¿Hay que llegar a 100? ¿Por qué?

LO QUE NOS LLEVAMOS HOY

Resumen de puntos clave

- Bastionar = reducir la superficie de ataque
- Medir (Lynis) → cambiar → volver a medir
- SSH: sin root, solo llave, 3 intentos… sin dejarte fuera
- UID 0 = root · SUID desconocido = puerta trasera
- Nginx: server_tokens off + cabeceras con always
- Fail2ban: auth.log → cuenta fallos → banea en nftables

Cerrar puertas y ventanas, quitar la llave maestra y medir.

LO SIGUIENTE

Parcial 2 y UD7 · Auditoría

- Parcial 2: UD4 a UD6
- UD7: mirar el edificio desde fuera
- Nmap y OpenVAS